

PhishGuard

Explainable machine-learning phishing-email detector

Author	Mohamed Ajab
Project type	Independent Python machine-learning experiment
Version	Reproducible synthetic-data baseline, September 2026
Repository	phishguard-ml
Data boundary	240 harmless synthetic messages; no private email or live links

This document explains what was built, why each part exists, how the experiment was run, what the results mean, and where the evidence stops.

1. Project summary

PhishGuard demonstrates a complete text-classification workflow without hiding the model behind a large framework. It joins an email subject and body, converts the text to TF-IDF word and phrase features, fits logistic regression, and reports both overall metrics and individual errors.

The project is educational. Its synthetic dataset is safe and reproducible, but easier than real inbox data. The result shows that the software pipeline behaves as intended; it is not a claim that the model would detect all real phishing.

Research question

Can a transparent linear text classifier separate phishing-like language from ordinary email in a controlled synthetic dataset, and what can its errors and feature weights tell us?



2. What was built

Measure	Result	What it means
Dataset builder	scripts/	Creates balanced fictional messages with a fixed seed and no active links.
Data validation	data.py	Checks required columns, both labels and non-empty input; then joins subject and body.
Feature pipeline	model.py	Uses lowercase TF-IDF unigrams and bigrams with a 3,000-feature limit.
Classifier	model.py	Fits class-balanced logistic regression with a fixed random state.
Evaluation	evaluation.py	Calculates accuracy, precision, recall, F1 and confusion-matrix counts.
Experiment	run_experiment.py	Creates a stratified 75/25 split, fits only on training data and saves predictions.
Dashboard	app.py	Classifies typed text and exposes terms associated with each class.

3. Why these methods

TF-IDF

Term frequency measures how often a word or phrase appears in a message. Inverse document frequency reduces the influence of terms that appear almost everywhere. The product gives each message a sparse numeric representation that works well as a readable text baseline.

Logistic regression

The classifier learns one coefficient per feature, adds the weighted values, and converts the result to a probability. Positive coefficients support the phishing label and negative coefficients support the legitimate label. This is fast, inspectable and suitable for a small learning project.

Pipeline

Keeping TF-IDF and the classifier in one scikit-learn pipeline ensures that fitting, prediction and evaluation apply the same transformation. It also reduces the chance of accidentally fitting the vocabulary on the test set.

4. Dataset and evaluation

The dataset contains 120 legitimate and 120 phishing-like messages. Subjects and bodies are generated from human-readable templates with changing names, topics, reference numbers and times. All examples are fictional and contain no live URLs.

A stratified split holds out 60 messages for testing while preserving the class balance. The random state is fixed at 42 so another person can reproduce the exact split. TF-IDF is fitted only on the 180 training messages.

Measure	Result	What it means
Training messages	180	Used to learn the vocabulary and classifier weights.
Held-out messages	60	Used once for the reported evaluation.
True positives	30	Phishing messages correctly flagged.
True negatives	30	Legitimate messages correctly allowed.
False positives	0	Legitimate messages incorrectly flagged.
False negatives	0	Phishing messages incorrectly allowed.

5. Reproduced results

Measure	Result	What it means
Accuracy	100%	All 60 held-out synthetic examples received the expected label.
Precision	100%	Every message flagged as phishing was labelled phishing in this dataset.
Recall	100%	Every labelled phishing message in the held-out set was found.
F1	100%	Precision and recall were both perfect on this controlled split.

A perfect score here should increase scrutiny, not confidence. The generator uses a small family of templates and related patterns appear in training and test data. Terms such as 'confirm your', 'refund', 'verify' and 'immediately' strongly support the phishing class, while 'meeting', 'library', 'project' and 'summary' support the legitimate class. Real attackers and legitimate organisations share vocabulary, so external performance will be lower.

6. Reproduce and verify

From the repository root on Windows PowerShell:

```
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python scripts/build_demo_dataset.py
python run_experiment.py
python -m pytest
streamlit run app.py
```

The latest verification completed four tests, regenerated all 240 messages, reproduced the 60-message held-out result and started the Streamlit dashboard successfully. The experiment saves metrics.json and predictions.csv under results/.

A five-minute demonstration

- Open data/messages.csv and compare one legitimate and one phishing-like example.
- Show model.py and explain how TF-IDF feeds logistic regression inside one pipeline.
- Run the experiment and connect each confusion-matrix count to precision and recall.
- Open the dashboard and classify one obvious message and one ambiguous message.
- Show the influential terms, then explain why correlation with a label is not proof of malicious intent.

7. Error costs and deployment thinking

The default threshold is 0.5, but a real system should choose a threshold from operational costs. Raising it usually reduces false positives but allows more phishing through. Lowering it usually finds more phishing but interrupts more legitimate mail.

Measure	Result	What it means
False positive	User disruption	A genuine urgent invoice or password notice could be quarantined.
False negative	Security exposure	A convincing phishing message could reach the user.
Probability	Model confidence	The value reflects this fitted model, not a guaranteed chance of phishing.
Human review	Escalation	Medium-confidence messages could be reviewed instead of automatically blocked.

8. Limitations and next study

- Synthetic templates do not reproduce the diversity, misspellings, multilingual text and campaign changes in real email.
- Related templates occur on both sides of the split, so the current evaluation tests recognition more than generalisation.
- The model ignores URLs, sender reputation, headers, attachments and domain information.
- Word features are vulnerable to obfuscation and novel phrasing.
- No probability calibration or threshold-cost study has been performed.

The next credible step is to freeze the model, obtain a legally reusable public dataset, and evaluate it without tuning on that test set. A campaign-level split, character n-gram baseline, calibration curve and threshold analysis would make the

study substantially stronger without making the code hard to explain.

9. Learning checklist

You should be able to explain:

- Why fitting TF-IDF on all messages before the split would leak information.
- What a coefficient means in logistic regression.
- Why accuracy alone is not enough when phishing is rare.
- How the confusion matrix produces precision and recall.
- Why 100% on this dataset is not a real-world performance claim.
- What extra signals a production email filter would need.

Small exercises

- Change the dashboard threshold from 0.5 to 0.7 and predict which errors will increase.
- Use only unigrams by changing `ngram_range` to `(1, 1)`, then compare the influential terms.
- Add a legitimate message containing urgent security language and inspect the prediction.

Evidence and references

Project evidence: `results/metrics.json`, `results/predictions.csv` and four automated tests. Method reference: scikit-learn documentation for `TfidfVectorizer` and `LogisticRegression` (<https://scikit-learn.org/stable/>). The included data is independently generated for this educational project.